Algorithm	*Data structure*	*Comparisons*	*Swaps*	*Time (in s)*
Selection sort	Array	3123750	2494	3.166
Selection sort	Linked list	3126250	2500	2.904
Bubble sort	Array	3123750	1579132	2.824
Bubble sort	Linked list	3123750	3123750	2.825
Quick sort	Array	32501	32501	0.029
Merge sort	Linked list	25093	0	0.023

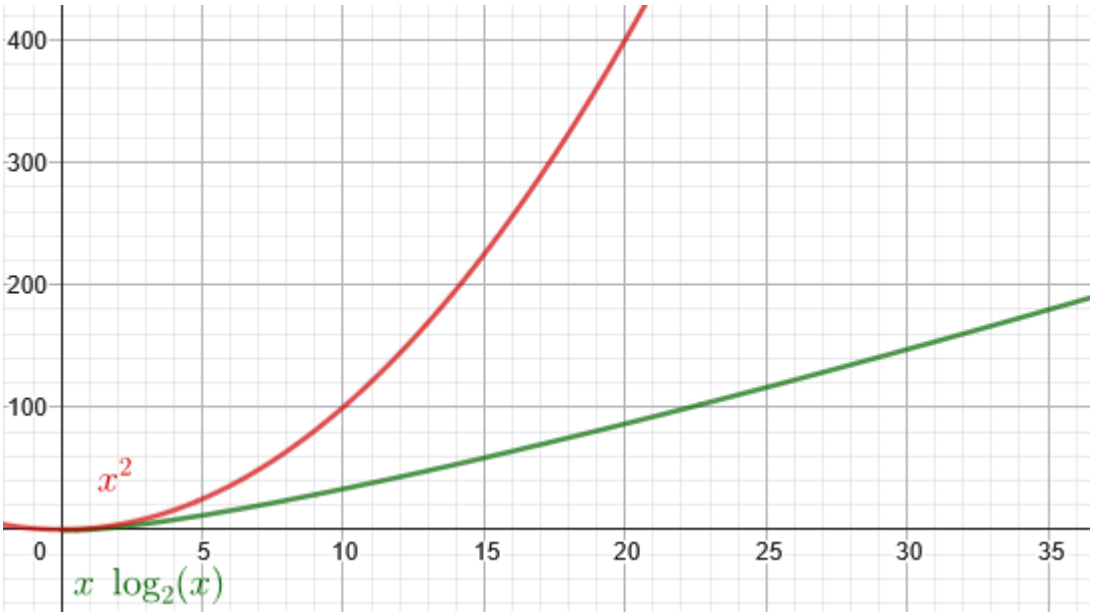
Selection sort is in theory harder on a linked list than an array. On an array you just have to increment pointers through the array and swap values between them. While on the linked list you have to move around in memory space following the pointer that points to the next node.

If you swap the nodes on a single linked list you have to seek through the list for the preceding node and change them. The two nodes then also have to swap next pointers.

You can avoid seeking by using a double linked list, but now you also have the previous node pointer to think about. The two nodes have to swap previous and next pointers and their neighbors have to have their previous/next pointer changed.

This can however be avoided by decoupling the content of a node from the node. Then the two nodes can just exchange the pointers to the data structure with the data. But you still have to make big jumps across memory space when navigating the list.

$O(n^2)$ vs. $O(n \log(n))$



Selection sort and bubble sort needed more than 2 seconds (in one case more than 3) to sort 2500 items while quick and merge sort used less than 0.030 seconds! This can be OK for rare usage on small datasets, but for hefty use on large datasets $O(n^2)$ is too slow.

Linear search vs. binary search

Search algorithm	*Data structure*	*Comparisons*
Linear search	Array/ linked list	1250 (logical average)
Binary search	Array	8

The binary search only needs 8 comparisons last time I ran it, while linear searches from submission 4 would have an amount of comparisons around n/2. If the element wasn't event in the array it would go through the entire array just for nothing!

The time taken by the binary search is so low that it comes back as 0, while linear searching would take around 0.029 seconds to go through the entire array. Of course the time will be lower if it finds what it's looking for at the start, but that is by pure chance.

I would like to highlight the similarities with binary search trees: When searching through both you start at a value that should ideally be near the median and you search on by constantly asking: is the value I'm looking for lower or higher than the one I'm at?